

Guía de estudio para el primer parcial

Este documento es una guía de los temas y bibliografía que vimos durante la cursada, para que tengan como referencia para el parcial. El alcance es la primera parte de la materia, que se da con Ruby. A todos los temas y bibliografía que mencionamos aquí (y que entran para el parcial) ya los discutimos en clase y es el material que vienen leyendo.

El objetivo de esta guía es que puedan tener una especie de “lista de verificación” para ver cómo van estudiando, y si se olvidaron de repasar algo. Si fueron leyendo el material durante la cursada, leer este documento les puede servir como un repaso en sí mismo para el parcial. Si no se les escapó ninguno de los temas que figuran a continuación, entonces están muy bien. Los temas no están listados en orden de importancia.

Por supuesto, las preguntas del parcial no van a incluir la reproducción mecánica del contenido (no va a estar enfocado en aspectos “memorísticos”, ni va a haber una única respuesta correcta a cada pregunta), pero sí van a requerir que hayan leído, que manejen los conceptos que vimos y que usen el vocabulario de manera apropiada.

Por supuesto, en caso de que les surjan dudas mientras están estudiando, siempre pueden consultar por Discord. También puede estar bueno si arman grupos de estudio para ver o repasar los temas.

Índice

Índice.....	1
Lista de temas / cosas que vimos.....	2
Repaso y profundización de temas previos.....	2
Esquemas no tradicionales para definir el comportamiento de los objetos.....	3
Metaprogramación.....	4
Apéndice: comentarios sobre la subclasificación.....	8
Lista de lecturas.....	10
Obligatorias.....	10
Opcionales.....	10

Lista de temas / cosas que vimos

Repaso y profundización de temas previos

- **¿Qué es un objeto? / Repaso del paradigma**

- Análisis crítico de distintas definiciones de "objeto".
- El concepto de "objeto" definido a partir de la noción de "mensaje".
- Propiedades deducibles a partir de esta definición: identidad, implementación interna y polimorfismo.

Este es un tema básico, pero muy útil para definir el vocabulario que usamos durante la cursada. Como es importante que se expresen con precisión durante el parcial, les podría venir bien un repaso sobre estos temas. Tienen los siguientes documentos:

- Diapositivas de la primera clase ( Algunas definiciones.pdf), que pueden encontrar [en el canal de material](#).
- Apunte [de lectura opcional] de "conceptos fundamentales de la programación con objetos" ( 1. Introducción a _conceptos fundamentales de la programación con objetos_.pdf ;  2. Conceptos fundamentales de la programación con objetos.pdf), que pueden encontrar [en la sección "Material adicional" del Drive](#).

- **Repaso de subclasificación**

- La subclasificación como herramienta técnica (la herencia, para "reusar código") *pero también* como herramienta de modelado conceptual.
- Algoritmo de búsqueda de método (*method lookup*).
- Limitaciones de la herencia simple.
- Interpretación y uso de *super*.

Para repasar esto, tienen los siguientes documentos:

- Diapositivas de la primera clase ( Clase 1 - Repaso herencia.pdf), que pueden encontrar en el canal de material.
- El apéndice "comentarios sobre la subclasificación" en este mismo documento, que repasa y entra en detalle sobre cosas que hemos mencionado en clase.
- El artículo *Mixin-based Inheritance* de Bracha & Cook, que pueden encontrar en la carpeta Bibliografía/Mixines en el Drive. En particular, la sección 2, que habla sobre herencia simple y compara *super* vs. *inner* en los lenguajes Smalltalk y Beta.
- El artículo *Traits: A Mechanism for Fine-grained Reuse* de Ducasse et al., que pueden encontrar en la carpeta Bibliografía/Traits en el Drive. En particular, la sección 2 hasta antes de la subsección 2.2, que habla de las limitaciones de la herencia simple, aunque también menciona herencia múltiple y mixines

(mezcla temas, aunque quizás no viene mal para repasar otras cosas también).

Esquemas no tradicionales para definir el comportamiento de los objetos

- **Mixines**

- Mixin como función (de superclase a clase) / “subclase abstracta”.
- Requisitos que establece el mixin sobre su superclase.
- Concepto de linearización. Interpretación de super.
- Comparación con herencia múltiple.

Para repasar esto, tienen los siguientes documentos y recursos:

- El artículo *Mixin-based Inheritance* de Bracha & Cook, que pueden encontrar en la carpeta Bibliografía/Mixines en el Drive. Pueden saltarse la sección 5.
- El artículo *Mixins in Strongtalk* [de lectura opcional], en la misma carpeta del Drive. En particular, la sección 2, que hace un resumen de lo que es un mixin.
- El video sobre herencia múltiple en C++, que pueden encontrar en el canal de material.
- El artículo *Traits: A Mechanism for Fine-grained Reuse* de Ducasse et al., que pueden encontrar en la carpeta Bibliografía/Traits en el Drive. En particular, las secciones 2 y 3. Este artículo entra muy en detalle sobre varias cosas, priorizaría los anteriores si no tienen tiempo suficiente para leer todo.

- **Traits**

- Definición de trait.
- Concepto de “flatening”, comparación con mixines. Interpretación de super.
- Separación de roles (clases vs. traits) según Ducasse et al.
- Reglas de composición, operadores, resolución de conflictos.

Para repasar esto, tienen los siguientes documentos:

- El artículo *Traits: A Mechanism for Fine-grained Reuse* de Ducasse et al., que pueden encontrar en la carpeta Bibliografía/Traits en el Drive. Lo más importante es la sección 1 (como introducción general) y el núcleo de la sección 4 (desde su inicio hasta el inicio de la subsección 4.1). En la primera parte de la sección 4.3 pueden encontrar la “ecuación” de las clases.

En el parcial no va a entrar ningún formalismo sobre los traits, pueden saltarse las definiciones estrictamente formales durante la lectura y no hay problema. Por otro lado, si conocen esas definiciones y quieren usarlas para responder alguna pregunta durante el parcial, es válido hacerlo (aunque no es una exigencia).

- El enunciado del TP 1 y su resolución. Supongo que este tema debería ser el que lleve menos tiempo de repaso, porque ya tienen el contexto de lo que hicieron para el TP!
- El resto de los artículos [de lectura opcional] en la carpeta Bibliografía/Traits en el Drive. La única idea de estos artículos es que sepan que el concepto de trait siguió evolucionando luego de la publicación del artículo *Traits: A Mechanism for Fine-grained Reuse*, que el mismo equipo de trabajo implementó traits con estado interno (finalmente no era una restricción esencial, sino más relacionada con los desafíos de implementación en Smalltalk), y sacar algunas ideas para profundizar en su implementación. No priorizaría estas lecturas para el parcial, siempre y cuando entiendan y sean conscientes de lo que mencionamos aquí. Por ejemplo, si en el parcial dicen “los traits *no pueden / no deben tener estado*”, entonces no están tomando en cuenta lo que dice este párrafo.
- **Closures**
 - Concepto de closure (expresión lambda abierta + contexto que la cierra).
 - Captura del contexto léxico y de `self`.
 - Ligadura del `return` (se liga de manera no local: *full closure* / `proc` en Ruby; se liga de manera local: `lambda` en Ruby). Casos en los que se produce un `LocalJumpError` en Ruby.

Lamentablemente, no hemos enviado bibliografía específica sobre esto. Vimos el tema de bloques a partir de la tercera clase, y luego profundizamos sobre el concepto de *closure* durante la clase en la que programamos el framework de testing (pueden encontrar el repositorio en el canal de material). En ese ejemplo, usamos un retorno no local para lograr que las pruebas no sigan ejecutándose luego de que falló una aserción (sin usar excepciones).

También explicamos que tener la posibilidad de ligar el return de esta manera es lo que habilita a poder implementar estructuras de control de manera no primitiva (ej. el `if`, pensemos en qué esperamos que suceda si tenemos un `return` dentro de un bloque de un `if`). Mencionamos también (al pasar) que también se pueden implementar excepciones (de manera no primitiva / *built-in*) con este mecanismo.

Metaprogramación

- **Introducción a la metaprogramación**
 - Concepto de metaprograma, ejemplos cotidianos.
 - Reflexión: introspección (lectura) vs. intercesión (escritura).
 - Modelo vs. metamodelo.
 - Sistemas reflexivos.
 - Distinción entre meta-niveles, consecuencias de diseño de juntarlos o separarlos.

Para repasar esto, tienen los siguientes documentos:

- Diapositivas de la segunda clase ([📄 Clase 2 - Intro Metaprogramación \(resumen\).pdf](#)), que pueden encontrar en el canal de material.
- El artículo *Concepts and Experiments in Computational Reflection* de Pattie Maes. De aquí sale la definición de sistema reflexivo, y otros conceptos (secciones 1 a 4). En la sección 5 se presenta la idea de reflexión procedural vs. reflexión declarativa. En la sección 7 se mencionan las distintas propiedades deseables de un sistema reflexivo orientado a objetos que explicamos en clase. También aparece el concepto de "meta-objeto", que está relacionado con la idea de *mirror*. Las secciones 1 a 4 son de lectura obligatoria, pero les recomiendo leer el artículo completo si tienen la posibilidad.
- Los ejercicios que hicieron (el del graficador y la reimplementación del *method lookup*). En ambos usaron introspección, y también apareció el concepto de *mirror*.
- El artículo [de lectura opcional] *Mirrors: Design Principles for Meta-level Facilities of Object-Oriented Programming Languages* de Bracha & Ungar. Aquí se describe el concepto de *mirror*. La idea está basada (en parte) en la primera propiedad deseable para un sistema reflexivo descrita en el artículo de Pattie Maes (la separación entre los niveles base y reflexivo de un sistema). En el artículo se postulan algunos principios de diseño para todo sistema orientado a objetos que sea reflexivo. Esto viene bien para tener clara la idea de la necesidad (a nivel diseño) de tener una buena separación entre el nivel base y el nivel meta de un sistema reflexivo.

- **Aspectos estructurales de los metamodelos**

(con esto nos referimos a las clases que conforman al metamodelo y sus relaciones)

- Metamodelo de Ruby, sus partes principales.
 - Relaciones conceptuales entre las distintas clases del metamodelo.
 - Generalidades de la API de metaprogramación de Ruby (ej. qué cosas se pueden hacer, poder distinguir cuándo un mensaje es reflexivo y cuándo no, cuándo es de introspección y cuándo no, etc.).
 - Cosas para mejorar en el metamodelo de Ruby.
- *Singleton classes* en Ruby. Metaclases (ej. en Smalltalk) como un caso particular de *singleton classes*.
- La necesidad de tener una jerarquía paralela entre clases y metaclases. Comunicación inter-nivel y compatibilidad ascendente/descendente.

Para repasar esto, tienen los siguientes documentos y recursos:

- Diapositivas de la tercera clase ([📄 Clase 3 - Singleton classes y otras cosas \(resumen\).pdf](#)), que pueden encontrar en el canal de material.
- El artículo *Safe Metaclass Programming*, que pueden encontrar en la carpeta Bibliografía/Metaprogramación del Drive. Este artículo explica en detalle el

problema de compatibilidad entre metaniveles, que es lo que justifica la jerarquía paralela entre clases y sus singleton classes / metaclases. El artículo además plantea algunos problemas con este tipo de soluciones. Las secciones más importantes son la 1 y la 2. La sección 3 da ejemplos en distintos lenguajes, y a partir de la sección 4 se propone un modelo para resolver los problemas que se plantean.

- La documentación oficial de Ruby, y las consultas que pueden hacer en el mismo ambiente para repasar cómo está conformado el metamodelo de Ruby.

- **Formas de representar y modificar el comportamiento dinámicamente**

- `Method` y `UnboundMethod`; diferencia conceptual. `bind` / `unbind` y sus restricciones.
- `instance_exec` vs. `module_exec` / `class_exec`. Razones por las que existen ambas variantes.
- Clases abiertas. Posibilidad de definir métodos nuevos con `define_method`. Cambio dinámico en el comportamiento de las instancias.

Para repasar esto, tienen los siguientes documentos y recursos:

- Diapositivas de la segunda y tercera clase (en ambos casos, llegando al final), que pueden encontrar en el canal de material.
- La documentación oficial de Ruby para cada una de esas clases y mensajes, y las pruebas que pueden hacer en el mismo ambiente.

Con respecto a esto, en el parcial no vamos a evaluar temas “sintácticos” de Ruby (ej. qué parámetros tienen esos mensajes, en qué orden van, etc.), sino más temas conceptuales (ej. por qué es necesario distinguir entre métodos ligados y no ligados, qué pasaría si solamente tuviéramos `instance_exec` pero no `class_exec`, etc.).

- **Comparación de metamodelos y maneras de entender a la programación**

- Generalidades sobre el metamodelo de Ruby comparado con el de Smalltalk y el de Java.
 - Smalltalk: (casi) todo se resuelve mediante el envío de mensajes (ej. el `if`); se programa en un ambiente vivo, modificando el sistema en tiempo de ejecución; tiene el concepto de metaclass reificado.
 - Java: muchas cosas se resuelven con construcciones que se van del paradigma (ej. los números no son objetos, los arrays no son colecciones, los operadores aritméticos y el `new` no son mensajes, etc.); metamodelo mínimo, las clases se representan únicamente con la clase `Class`, luego los “métodos estáticos” no pueden ser realmente métodos (y entonces no hay polimorfismo, no hay `this`, etc.).
 - Ruby, de alguna manera, está “en el medio” del espectro.
¿Qué gana y qué pierde cada lenguaje?

- Enfoque centrado en los “lenguajes” de programación vs. en los “sistemas/ambientes” de programación. Programa como artefacto lingüístico, estático, vs. como proceso en ejecución, dinámico.
- Resolución de problemas “de manera paradigmática” (usando solamente los conceptos del paradigma) vs. “de manera sintáctica” (agregando sintaxis específica).
- Posibilidad del lenguaje de darnos una sintaxis flexible, que permita expresarnos como queremos sin agregar palabras reservadas (ej. lo que sucede con Ruby).

Todos estos son conceptos que fuimos explorando durante la cursada, pero lamentablemente no hemos podido hacer una selección bibliográfica sobre estos temas aún. Para consultar, tienen las diapositivas de la clase ([Smalltalk y otros metamodelos.pdf](#)) en el canal de material, aunque no es necesario que estudien la parte histórica para el parcial.

A los temas que tienen que ver con la idea de extensiones sintácticas vs. usar los elementos del paradigma los hemos hablado cuando discutimos sobre la idea de la “interfaz de usuario” de un metaprograma (ej. lo que hicimos en el TP 1) en comparación con el modelo interno. También vimos ejemplos usando la sintaxis que ofrece Ruby para las clases (`class X; end`) en comparación con usar los elementos básicos del paradigma para hacer exactamente lo mismo (`X = Class.new`); Ruby habilita el uso de ambas opciones.

Lo que es más importante es que hayan podido desarrollar un criterio durante la cursada para poder hacer las distinciones que mencionamos (sobre los distintos enfoques y manera de resolver los problemas), y eventualmente posicionarse con respecto a ellas de manera informada¹.

Una lectura que puede ser interesante (no obligatoria, y que no habíamos mencionado antes), es el artículo *The Structure of a Programming Language Revolution*, de Richard Gabriel (pueden encontrarlo en la carpeta de Material Adicional en el Drive). No es necesario que lo lean para el parcial, sólo lo menciono como lectura relacionada, por si les interesa el tema.

¹ Lo importante no es tanto cómo se posicionen, sino con qué argumentos y justificaciones. En la segunda mitad de la cursada vamos a poner el foco en otros enfoques, más alineados con la noción de “lenguaje” y no tanto con la de “sistema”.

Apéndice: comentarios sobre la subclasificación

En clase hemos visto la siguiente heurística para saber si corresponde que una clase X sea subclase de otra clase Y:

“todo X es un Y”

(en tanto y en cuanto se comporta como tal, es decir, respeta el protocolo)

Una diferencia que comentamos fue con respecto a la necesidad de decir “*todo* _ es un”, en lugar de simplemente decir “es un” (que es lo que normalmente se dice en el material disponible en la web al respecto de la herencia). Sin embargo, decir “es un” confunde una relación entre clases con una relación entre una clase y sus instancias.

Comentario al margen: esa confusión está relacionada con algo que hemos comentado en clase, y es que hay una confusión entre clases y objetos. A veces hasta parece que en lugar de programar “orientado a objetos” programamos “orientado a clases”². Por supuesto, esto sucede en los lenguajes basados en clases y es consecuencia, en parte, de las herramientas que usamos y de la manera en la que pensamos a la programación. Es decir, si lo que nos importa es el programa como artefacto estático, entonces tendemos a pensar en términos de clases porque los objetos sólo existen cuando el programa se ejecuta.

Pero bueno, volviendo a la idea del “es un”, dejo aquí un extracto del borrador de un apunte que estoy preparando para Objetos 1 sobre subclasificación, en donde estamos describiendo lo que ocurre en el dominio de interés sobre el cual estamos trabajando, y que luego representamos en nuestro programa mediante clases (que representan conceptos) y objetos (que representan cosas).

Tenemos, entonces, dos tipos de relaciones en nuestros dominios:

1. *La relación de generalización* (entre conceptos), que es la que se corresponde con la relación subclase-superclase en nuestros programas (en donde reificamos los conceptos como clases). Esta relación también es conocida como “relación de subsunción” o como “subordinación conceptual”. Sería el equivalente a la *inclusión* en teoría de conjuntos³:

$$\text{ConceptoEspecífico} \subseteq \text{ConceptoGeneral}$$

² Una vez que somos conscientes de esto, es algo que empieza a aparecer en muchos lugares. Por ejemplo, la formulación de la “S” de los principios SOLID es “*una clase debería tener una única razón para cambiar*” (traducido, literalmente, a partir [del capítulo 8 del libro de Robert Martin](#)).

Una solución obvia, entonces, para aplicar este “principio” (entendido de esa manera) sería tener una cadena de subclases en la cual cada una tenga sólo una razón para cambiar, y que nuestros objetos sean instancias de la clase que está más abajo en esa jerarquía. En ese caso, el principio literalmente se cumple... pero nuestro objeto sigue teniendo intactas todas sus responsabilidades, y su cohesión sigue siendo tan baja como lo era antes!

³ Formalmente, decimos que la *extensión* de un concepto es el conjunto de todas las cosas que caen bajo ese concepto. En estos ejemplos, cuando hablamos del concepto “como conjunto”, en realidad nos estamos refiriendo a su *extensión*.

2. *La relación entre una cosa y un concepto bajo el cual cae*, que es la que se corresponde con la relación clase-instancia en nuestros programas. Esta relación también es conocida como “ejemplificación” (ej. *pepita es un ejemplo de golondrina*) o “es un” (ej. *pepita es una golondrina*). En teoría de conjuntos, esta sería la relación de *pertenencia*:

$$\text{individuo} \in \text{Concepto}$$

El vínculo entre ambas relaciones tiene que ver con que *para que un concepto sea más general que otro, todas las cosas que caen bajo el concepto específico deben caer bajo el concepto general*. Por ejemplo, si podemos afirmar que “toda caja de ahorros es una cuenta bancaria”, entonces el concepto de cuenta bancaria es más general que el de caja de ahorros. En notación de conjuntos:

$$\text{ConceptoEspecífico} \subseteq \text{ConceptoGeneral}$$

$$\Leftrightarrow$$

$$(\forall \text{ individuo} \in \text{ConceptoEspecífico}) \text{ individuo} \in \text{ConceptoGeneral}$$



Vemos, entonces, que la relación “es un” es en realidad la relación entre una instancia y su clase (o, mejor dicho, entre una cosa y un concepto bajo el cual cae, que están representados en nuestro programa como una instancia y una clase), no la relación entre dos clases. En cambio, si decimos “todo _ es un”, entonces sí estamos hablando de una relación entre clases (en la que el “todo” nos sirve para cuantificar a las instancias, sin referirnos a una en particular).

Lista de lecturas

Obligatorias

- **Maes, P. (1987).** *Concepts and Experiments in Computational Reflection*. OOPSLA '87. Secciones 1 a 4, aunque es muy recomendable leer el artículo completo.
- **Bouraqadi-Saâdani, N., Ledoux, T. & Rivard, F. (1998).** *Safe Metaclass Programming*. OOPSLA '98. La sección 2 es obligatoria, pero es recomendable leer las secciones 1-3.
- **Bracha, G. & Cook, W. (1990).** *Mixin-based Inheritance*. OOPSLA/ECOOP '90. La lectura es obligatoria, salvo por la sección 5.
- **Ducasse, S., Nierstrasz, O., Schärli, N., Wuyts, R. & Black, A. P. (2006).** *Traits: A Mechanism for Fine-grained Reuse*. ACM TOPLAS 28(2), 331–388. La sección 4 es la referencia sobre traits, aunque es recomendable leer el paper completo o hacer una selección de las partes que resulten más apropiadas para los temas que quieran repasar.

Opcionales

- **Bracha, G. & Ungar, D. (2004).** *Mirrors: Design Principles for Meta-level Facilities of Object-Oriented Programming Languages*. OOPSLA '04. Repaso del concepto de *mirror* y de la separación nivel base / nivel meta.
- **Bak, L., Bracha, G., Grarup, S., Griesemer, R., Griswold, D. & Hölzle, U. (2002).** *Mixins in Strongtalk*. ECOOP '02 Workshop on Inheritance. Repaso del modelo de mixins (recomendada la sección 2).
- **Bergel, A., Ducasse, S., Nierstrasz, O. & Wuyts, R. (2006).** *Stateful Traits*. ISC 2006, LNCS 4406, 66–90. Extensión del modelo de traits para agregarles estado.
- **Tesone, P., Ducasse, S., Polito, G., Fabresse, L. & Bouraqadi, N. (2020).** *A new modular implementation for Stateful Traits*. Science of Computer Programming 195, 102470. Implementación modular de traits con estado vía metaclasses especializadas.